Database Design

This document covers the SQLite database schema, the data access layer, and the programming concepts applied in the database module.

Overview

The system uses a single SQLite database file (`signin_data.db`) accessed by both the kiosk and the web dashboard. All schema definitions, queries, and data access functions live in `kiosk_signin/database.py`.

Schema

The database consists of eleven tables:

members

The central table. Stores both children (members) and staff. The `person_type` column distinguishes between them (`"member"` or `"staff"`). Staff members reuse the `school` column to store their role/position.

Column	Type	Purpose
id	INTEGER	Primary key, auto-increment
first_name	TEXT	Required
last_name	TEXT	Required
date_of_birth	TEXT	Date string (YYYY-MM-DD)
school	TEXT	School name (members) or role (staff)
mornings_days	TEXT	Comma-separated days (e.g. "Mon,Tue,Wed")
afternoons_days	TEXT	Comma-separated days
needs_pickup	INTEGER	Boolean (0/1)
activity	TEXT	After-school activity name
activity_time	TEXT	Time of activity

activity_address	TEXT	Address of activity
transport_required	INTEGER	Boolean (0/1)
activity_days	TEXT	Comma-separated days
pin	TEXT	Optional PIN for sign-in verification
person_type	TEXT	"member" or "staff"
created_at	TEXT	Auto-set to local datetime on insert
updated_at	TEXT	Set on update

caregivers

Each caregiver is linked to a member via `member_id`. A member can have one caregiver record. The `notify_on` column controls email notification preferences.

Column	Type	Purpose
id	INTEGER	Primary key
member_id	INTEGER	Foreign key to members, CASCADE
full_name	TEXT	Caregiver's name
phone	TEXT	Phone number
street_address	TEXT	Address line
suburb	TEXT	Suburb
city	TEXT	City
email	TEXT	Email for notifications
emergency_contact_name	TEXT	Emergency contact
emergency_contact_phone	TEXT	Emergency phone
relationship	TEXT	e.g. "Parent/Guardian"
notify_on	TEXT	"NONE", "IN", "OUT", or "BOTH"

sign_ins

Records every sign-in and sign-out event. The `signature` column stores raw stroke data as a serialised Python list (bytes).

Column	Type	Purpose
id	INTEGER	Primary key
member_id	INTEGER	Foreign key to members, CASCADE
direction	TEXT	"IN" or "OUT" (enforced with CHECK)
timestamp	TEXT	Auto-set to local datetime
notes	TEXT	Optional notes entered during sign-in
signature	BLOB	Serialised stroke coordinates

A member's "current"/most-recently-used signature is never stored separately — it's computed on demand by `database.get_last_signature(member_id)`, which finds the most recent row for that member with a non-null signature. This follows the same "compute on the fly" pattern as `get_last_sign()`/`get_today_last_sign()`/`get_allowed_direction()` below rather than caching state on the `members` row.

admins

Stores the admin account. Only one admin account exists by default.

Column	Type	Purpose
id	INTEGER	Primary key
username	TEXT	Unique username
password_hash	TEXT	Salted PBKDF2 hash of the password (`werkzeug.security`). Legacy unsalted SHA-256 hashes from older installs are upgraded automatically on the next successful login

dashboard_users

Login accounts for the **web dashboard** -- separate from the kiosk's single-row `admins` table above. Supports any number of accounts, each with its own name, hashed password, and role.

Column	Type	Purpose
id	INTEGER	Primary key
name	TEXT	Unique login name
pin_hash	TEXT	Salted hash of the password (`werkzeug.security`); column name kept as `pin_hash` for backward compatibility even though it now stores a password, not a numeric PIN
role	TEXT	`"admin"` or `"staff"` (enforced with CHECK)
created_at	TEXT	Auto-set to local datetime on insert
member_id	INTEGER	Optional FK to `members.id`. Links a staff login to their own staff record so they can view/download their own timesheet on the "My Timesheet" page. `NULL` for admin accounts and for staff logins an admin hasn't linked yet.
failed_attempts	INTEGER	Count of consecutive failed logins for this account, defaults to 0. Reset to 0 on a successful login.
locked_until	TEXT	If set and in the future, this account is temporarily locked out of logging in, even with the correct password. `NULL` when not locked.

`admin` accounts get full dashboard access; `staff` accounts can only use the web Sign In/Out page (and their own timesheet, if `member_id` is set). One `admin`/`1234` row is seeded on first run so the dashboard always has a working login -- see 09-authentication-and-security.md for the hashing scheme, password rules, lockout behavior, session model, and why this table is kept separate from `admins` rather than extending it.

`member_id`, `failed_attempts`, and `locked_until` were added after the table already existed in deployed installs, via the same additive `ALTER TABLE ... ADD COLUMN` migration pattern used

throughout `init_db()` (see Schema Migration below) -- existing rows get `NULL`/`0` defaults automatically, and no manual migration step is ever needed.

run_groups and run_members

Run sheets group members for batch sign-in (e.g. a school bus run). `run_members` is a junction table implementing the many-to-many relationship.

run_groups:

Column	Type	Purpose
id	INTEGER	Primary key
name	TEXT	Unique run name

run_members:

Column	Type	Purpose
run_id	INTEGER	Foreign key to run_groups
member_id	INTEGER	Foreign key to members

Composite primary key on `(run_id, member_id)`.

Run sheets can be fully managed (created, renamed, and deleted, with membership edited) from both the kiosk's Run Sheets screen and the web dashboard's Run Sheets pages -- see 06-flask-web-dashboard.md for why the dashboard uses id-keyed `create_run()`/`update_run()` functions rather than the kiosk's name-keyed `upsert_run()`. Every create, rename, and delete is recorded in `audit_log` (`RUN_CREATED`, `RUN_UPDATED`, `RUN_DELETED`), regardless of which side made the change.

audit_log

Immutable log of all system events (sign-ins, profile changes, exports, dashboard logins).

Column	Type	Purpose
id	INTEGER	Primary key
event_type	TEXT	e.g. "SIGN_IN", "MEMBER_CREATED"
description	TEXT	Human-readable description
timestamp	TEXT	Auto-set to local datetime

alerts

Messages that pop up when particular people are being signed in or out, on both the kiosk and the web Sign In / Out page. Created from the kiosk Admin Panel ("Member / Run Alerts") or the dashboard's Alerts page.

Column	Type	Purpose
id	INTEGER	Primary key
scope	TEXT	`"all"`, `"run"`, or `"member"` (enforced with CHECK)
target_id	INTEGER	`run_groups.id` for `run`, `members.id` for `member`, `NULL` for `all`
message	TEXT	Required. The text shown in the popup
created_at	TEXT	Auto-set to local datetime on insert
expires_at	TEXT	When the alert stops appearing. `NULL` means it never expires

`target_id` is not a foreign key, because it points at a different table depending on `scope`. Deleting a run sheet or person therefore leaves their alerts in place, and the Alerts page shows the target as "(deleted run)" or "(deleted person)". `get_alerts_for_members(member_ids)` returns every unexpired alert that applies to any of the given people: `all` alerts, `member` alerts for those ids, and `run` alerts for any run sheet they belong to. Creating and deleting alerts is recorded in `audit_log` (`ALERT_CREATED`, `ALERT_DELETED`).

Entity Relationship Summary

```
members  1 --- *  sign_ins
members  1 --- 1  caregivers
members  * --- *  run_groups       (via run_members junction table)
members  1 --- *  dashboard_users  (optional member_id link, for My Timesheet)
alerts   * --- 1  members or run_groups  (scope + target_id, not a foreign key)
```

The foreign keys on `sign_ins`, `caregivers`, and `run_members` use `ON DELETE CASCADE`, so deleting a member automatically removes their sign-in records, caregiver details, and run sheet assignments. `dashboard_users.member_id` has no `ON DELETE` action, so `delete_member()` first sets `member_id` to `NULL` on any linked dashboard login (in the same transaction) before deleting the member. The login itself is kept; it just loses its My Timesheet link.

Programming Concepts Applied

Connection Management

Each function opens a fresh connection with `get_conn()` and uses Python's `with` statement (context manager) for automatic transaction handling. SQLite connections in Python commit on successful exit from the `with` block and roll back on exception.

```
def get_conn():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row    # rows behave like dicts
    conn.execute("PRAGMA journal_mode=WAL")
    conn.execute("PRAGMA foreign_keys=ON")
    return conn
```

WAL Mode

Write-Ahead Logging (`PRAGMA journal_mode=WAL`) allows concurrent readers and a single writer without blocking. This is essential because the kiosk and dashboard both access the same database file simultaneously from different threads.

Row Factory

Setting `conn.row_factory = sqlite3.Row` means query results are returned as `sqlite3.Row` objects that support both index-based and key-based access. All functions convert these to plain dictionaries with `dict(row)` before returning, keeping the rest of the codebase independent of SQLite internals.

Upsert Pattern

The `upsert_member()` function checks whether the data dictionary contains an `id` key. If it does, it performs an UPDATE; otherwise, it performs an INSERT and returns the new row's ID via `cursor.lastrowid`. This avoids requiring separate create and update functions.

Schema Migration

New columns are added using `ALTER TABLE ... ADD COLUMN` wrapped in try/except blocks. If the column already exists, the `ALTER TABLE` raises an error which is silently caught. This allows the schema to evolve without requiring a separate migration tool.

```
try:
    conn.execute("ALTER TABLE members ADD COLUMN person_type TEXT DEFAULT 'member'")
except Exception:
    pass
```

Seed Data

`init_db()` checks whether key tables are empty (e.g. `SELECT COUNT(*) FROM admins`) and inserts default data if so. This provides a working system out of the box with sample members, a default kiosk admin account, and a default dashboard login (`dashboard_users`).

Parameterised Queries

All queries use parameterised placeholders (`?`) rather than string interpolation to prevent SQL injection. This is a fundamental security practice applied consistently throughout the module.

Audit Logging

The `log_event()` function records an entry in the `audit_log` table. It accepts an optional `conn` parameter so it can participate in the same transaction as the operation being logged, ensuring atomicity. If no connection is provided, it opens its own.

Direction Validation

The `get_allowed_direction()` function implements a small state machine for the sign-in/out flow, shared by members and staff alike:

1. No signs today: allowed direction is `IN`
2. Last sign today was `IN`: allowed direction is `OUT`
3. Last sign today was `OUT`: allowed direction is `IN`

There is no time-of-day cutoff and no daily session limit — a person can sign in and out as many times as they like, at any time of day. The function only ever alternates between `IN` and `OUT` based on the most recent event; it never returns `None`. "Most recent" means latest by `timestamp` (then `id` as a tie-break), not by insertion order -- a record back-dated from the dashboard's Logs page is inserted with a higher `id` but an earlier time, and must not count as the person's latest event. `get_last_sign()` and `get_currently_signed_in()` use the same ordering.

Dashboard Corrections

The dashboard's Logs page edits sign records with `update_sign_time()`, `add_sign()` and `delete_signs()`. Because IN/OUT visits are rebuilt from time order rather than stored, `get_sign_neighbours(sign_id)` returns the person's records immediately before and after a given one; the dashboard only accepts a new/edited time strictly between those, so a correction can never re-pair records into different visits. An earlier version locked a person out after a second ("PM") sign-out each day — that lockout was removed so both members and staff can freely sign in/out whenever they need to (e.g. staff popping out and back multiple times in a shift).